Ch1 Introduction (Part 1)

Server Side Fundamentals

What is the meaning of server?

- A server is a computer or a software system that provides services, or resources to other computers or clients over a network. In the context of web development, a server typically refers to a computer program or hardware device that hosts websites, web applications, or other resources accessible over the internet or an intranet.
- The primary role of a server is to respond to requests from clients (such as web browsers) by providing the requested resources, or performing computations.
- Servers are designed to operate continuously, listening for incoming requests and responding to them promptly and efficiently.
- Servers often run specialized software, such as web server software, application servers, database servers, email servers, and file servers.

What is server side development?

- Refers to software development for application in which majority of processing occurs on one or more servers (and not on the client's machine).
- In web development, the server-side code is responsible for processing requests from clients (such as web browsers), executing application logic, accessing databases, and generating dynamic content to be sent back to the client.

Introduction to server side

- Server-side code is used by most large-scale websites to dynamically display different data when needed, generally pulled out of a database stored on a server and sent to the client to be displayed via some code (e.g. HTML and JavaScript).
- It allows you to tailor website content for individual users. Dynamic sites can highlight content that is more relevant based on user preferences and habits. It can also make sites easier to use by storing personal preferences and information — for example reusing stored credit card details to streamline subsequent payments.
- It can even allow interaction with users of the site, sending notifications and updates via email or through other channels.

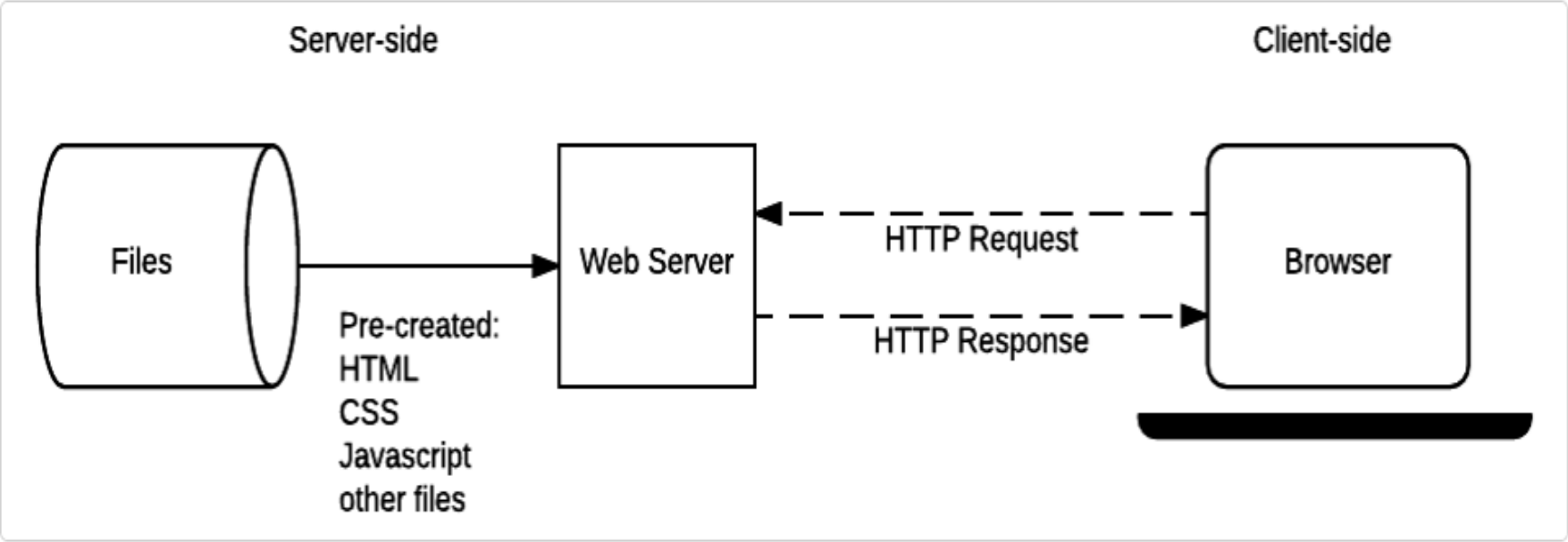
What is server-side website programming?

- Web browsers communicate with web servers using the HyperText Transfer Protocol (HTTP). When you click a link on a web page, submit a form, or run a search, an HTTP request is sent from your browser to the target server.
- Web servers wait for client request messages, process them when they arrive, and reply to the web browser with an HTTP response message. The response contains a status line indicating whether or not the request succeeded.
- The body of a successful response to a request would contain the requested resource (e.g. a new HTML page, or an image), which could then be displayed by the web browser.

Static sites

- Static web pages remain constant over time. They are boring: no account management, no statefulness (statefulness mainly related to tracking the state or context of user interactions).
- The diagram below shows a basic web server architecture for a static site (a static site is one that returns the same hard-coded content from the server whenever a particular resource is requested). When a user wants to navigate to a page, the browser sends an HTTP "GET" request specifying its URL.
- The server retrieves the requested document from its file system and returns an HTTP response containing the document and a success status. If the file cannot be retrieved for some reason, an error status is returned.

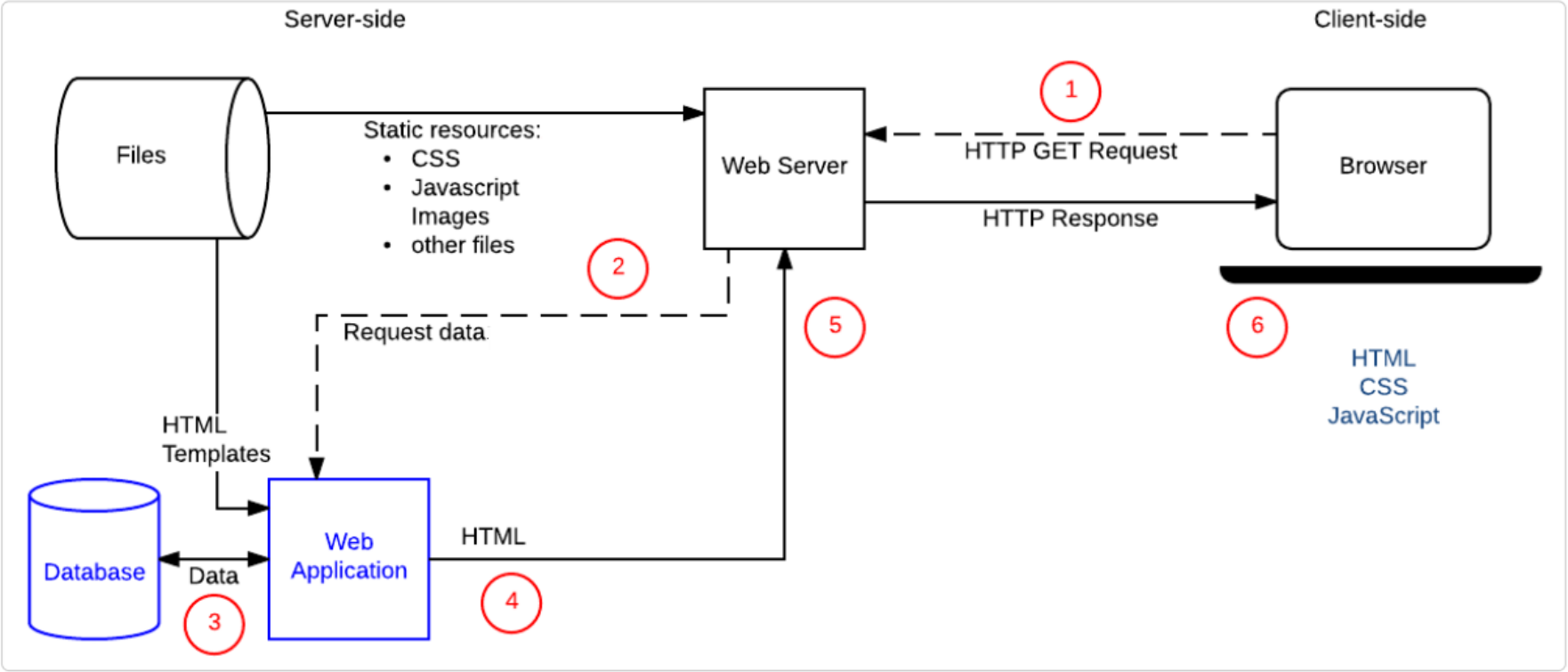
Static sites



Dynamic sites

- Dynamic web pages are built with server software that generates HTML or DOM subtrees on the fly!
- Dynamic web pages come in two flavors: **Server-side dynamic**: Client gets an entire new HTML page on each request. **Client-side dynamic:** Client gets DOM subtrees on each request which requires the use of JavaScript to modify the DOM on the fly in the client.
- Most of the code to support a dynamic website must run on the server. Creating this code is known as "server-side programming" (or sometimes "back-end scripting").
- The diagram below shows a simple architecture for a dynamic website. As in the previous diagram, browsers send HTTP requests to the server, then the server processes the requests and returns appropriate HTTP responses.

Dynamic sites

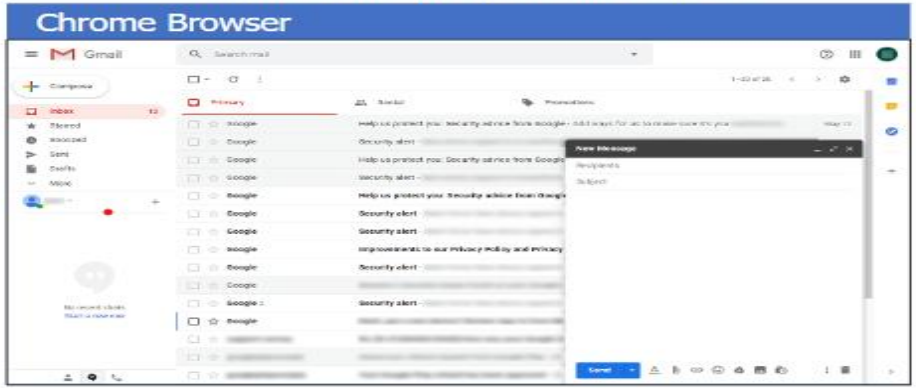


Dynamic sites

- Requests for static resources are handled in the same way as for static sites (static resources are any files that don't change — typically: CSS, JavaScript, Images, pre-created PDF files, etc.).
- Requests for dynamic resources are instead forwarded (2) to server-side code (shown in the diagram as a Web Application). For "dynamic requests" the server interprets the request, reads required information from the database (3), combines the retrieved data with HTML templates (4), and sends back a response containing the generated HTML (5,6).

How the web works?

1. Front end – what the user sees



2. Network – information travel

3. Back end – runs on Google servers

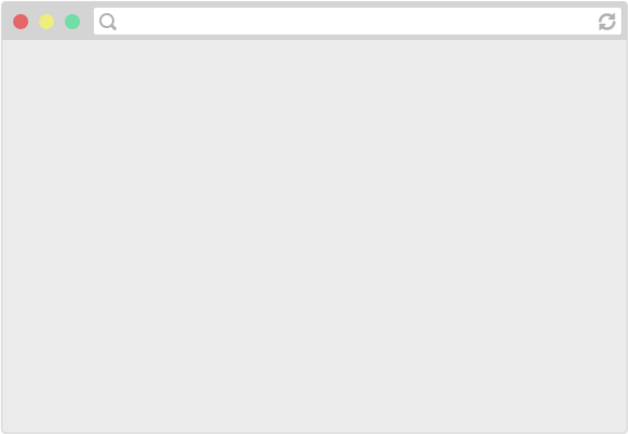
4. Storing and processing 50,000 emails for 1 billion and more users

5. Searching 50,000 emails in under 1 second

How the web works?

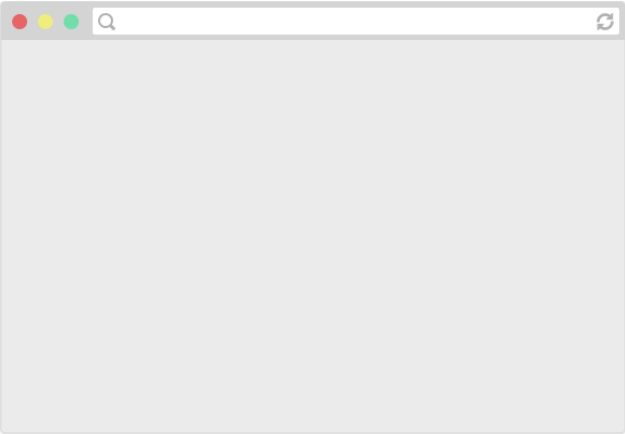
- "Full stack" = frontend and backend
- Frontend: runs in web browser (Chrome, Safari, Firefox, etc.)
- Backend: runs on servers
- The Web is drawn from many sub-disciplines of computer science, so you will find interesting overlap with many other previous and upper level CS courses.

How do web pages work?



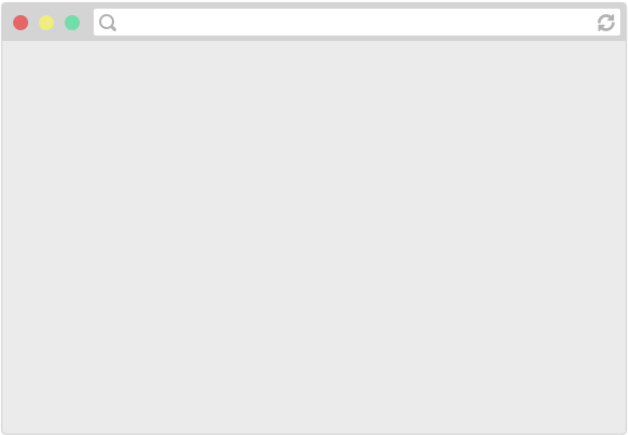
Browsers are applications that can display web pages.
E.g. Chrome, Firefox, Safari, Internet Explorer, Edge, etc.

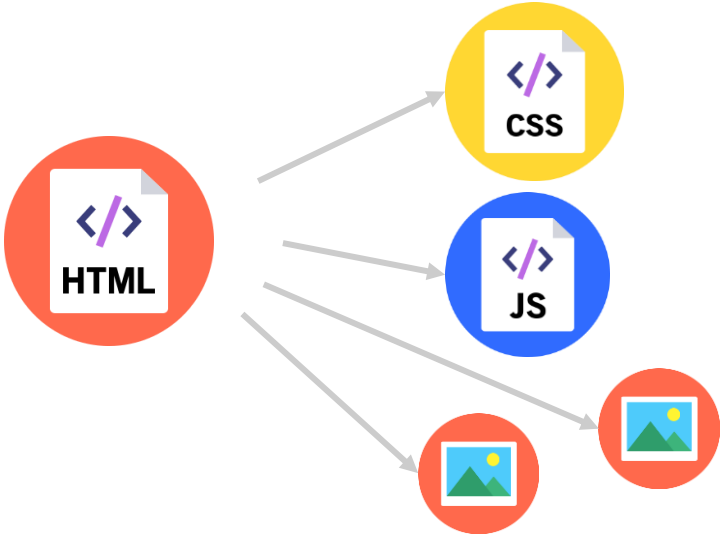
How do web pages work?



Web pages are written in a markup language called **HTML**, so browsers display a web page by reading and interpreting its HTML.

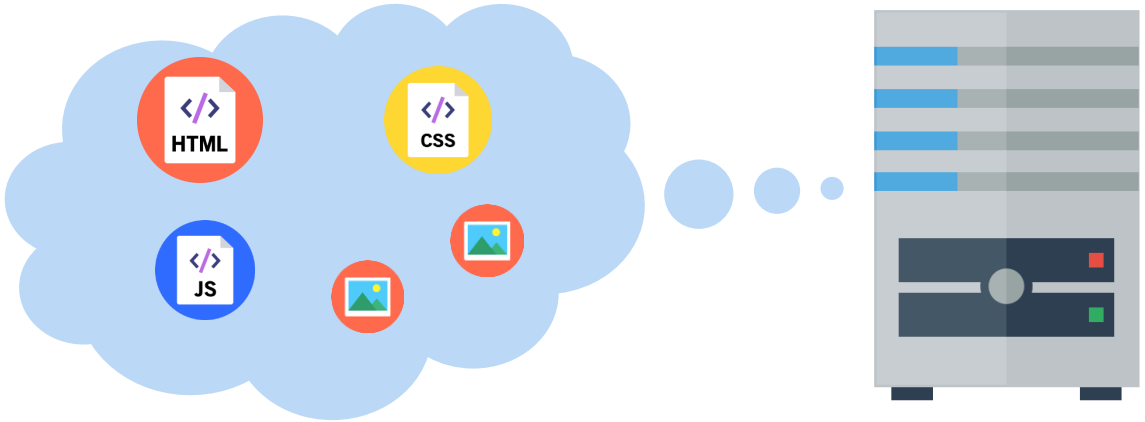
How do web pages work?





The HTML file might link to other resources, like images, videos, as well as **JavaScript** and **CSS** (stylesheet) files, which the browser then also loads.

How do web pages work?

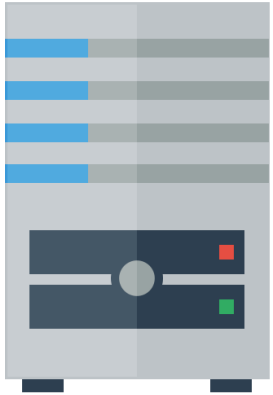


A **web server** is a program running on a computer that delivers web pages in response to requests.
It either stores or generates the web page returned.

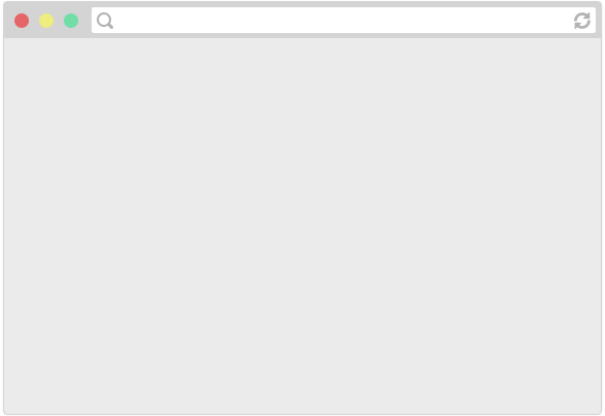
How do web pages work?

1. You type in a URL, which is the address of the HTML file on the internet.

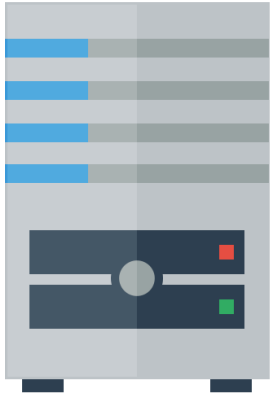
How do web pages work?

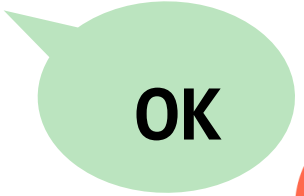


2. The browser asks the web server that hosts the document to send that document.

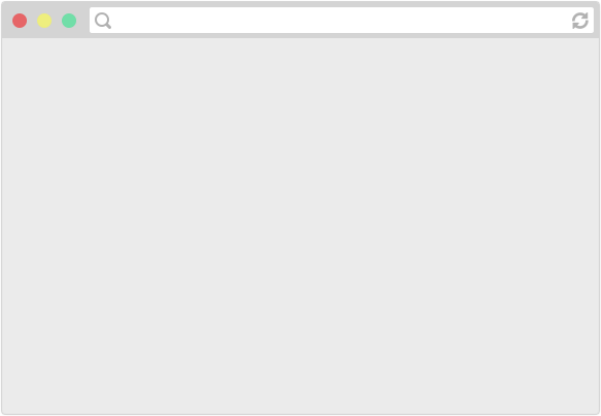


How do web pages work?





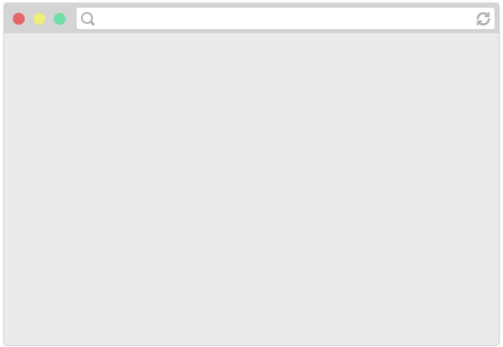
3. The web server responds to the browser with HTML file that was requested.

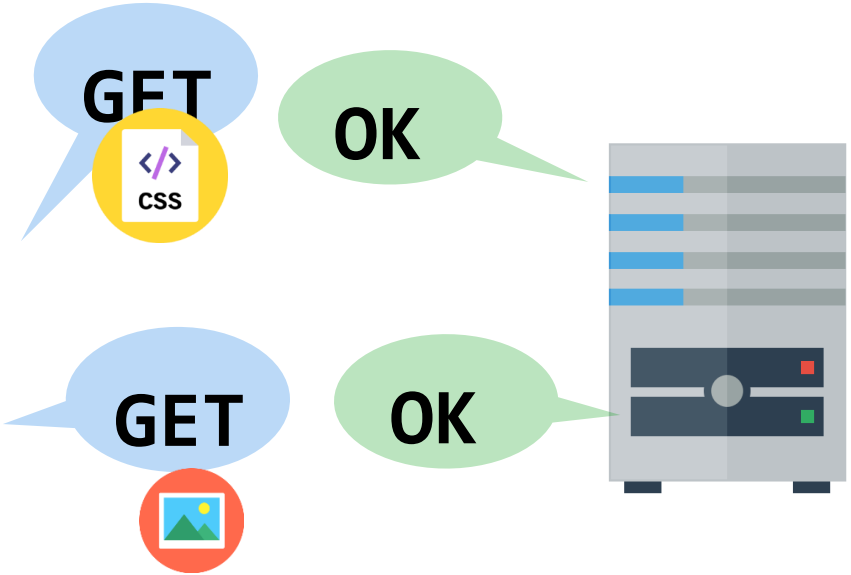


How do web pages work?

4. The browser reads the HTML, sees the embedded resources and asks the server for those as well.

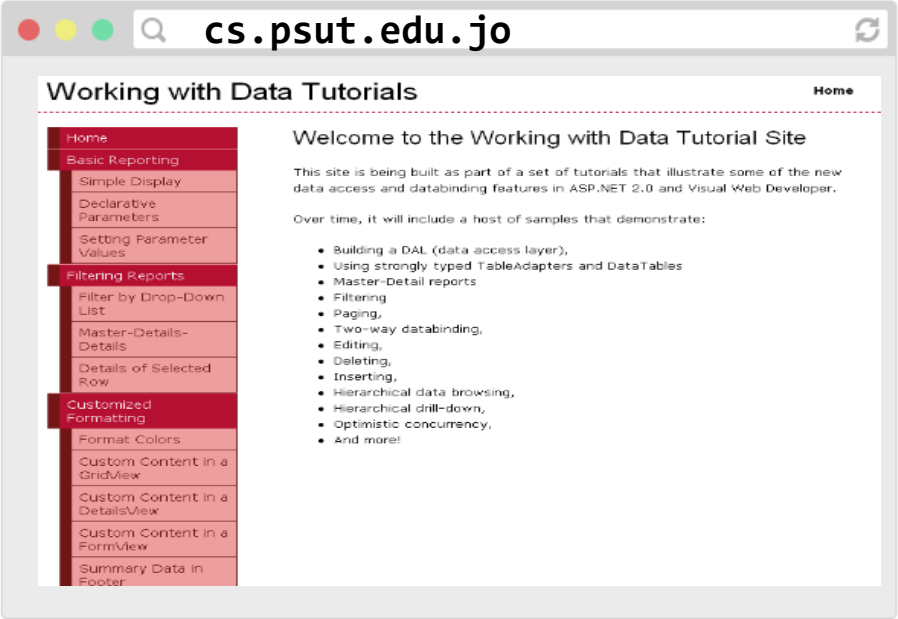
• • •





How do web pages work?

5. The web page is loaded when all the resources are fetched and displayed.



Ch1 Introduction (Part 2)

Server Side Fundamentals

Server-side vs. client-side programming

- Code running in the browser is known as client-side code and is primarily concerned with improving the appearance and behavior of a rendered web page. This includes selecting and styling UI components, creating layouts, navigation, etc.
- By contrast, server-side website programming mostly involves choosing which content is returned to the browser in response to requests. The server-side code handles tasks like validating submitted data and requests, using databases to store and retrieve data and sending the correct data to the client as required.

Server-side vs. client-side programming

- Client-side code is written using HTML, CSS, and JavaScript — it is run inside a web browser and has little or no access to the underlying operating system (including limited access to the file system).
- Server-side code can be written in any number of programming languages — examples of popular server-side web languages include PHP, Python, Ruby, C#, and JavaScript (NodeJS). The server-side code has full access to the server operating system and the developer can choose what programming language they wish to use.

Server-side vs. client-side programming

- Developers typically write their code using web frameworks. Web frameworks are collections of functions, objects, rules and other code constructs designed to solve/simplify common problems, and speed up development.
- Frameworks are designed to aid in the development of web applications by providing a structured approach to building and organizing code, handling common web development tasks, and promoting code reusability and maintainability. Web frameworks typically offer a collection of libraries, modules, and tools, that streamline the process of creating web applications.

Server-side vs. client-side programming

- Client-side web frameworks simplify layout and presentation tasks.
- While server-side web frameworks provide a lot of common web server functionality that you might otherwise have to implement yourself (e.g. support for sessions, support for users and authentication, easy database access, etc.).

What can you do on the server-side?

- Server-side programming is very useful because it allows us to efficiently deliver information tailored for individual users and thereby create a much better user experience.
- Companies like Amazon use server-side programming to construct search results for products, make targeted product suggestions based on client preferences and previous buying habits, simplify purchases, etc.
- Banks use server-side programming to store account information and allow only authorized users to view and make transactions.
- Some of the **common uses and benefits of server-side programming** are:

What can you do on the server-side?

1. **Efficient storage and delivery of information:**

- Imagine how many products are available on Amazon, and imagine how many posts have been written on Facebook? Creating a separate static page for each product or post would be completely impractical.
- Server-side programming allows us to instead store the information in a database and dynamically construct and return HTML and other types of files (e.g. PDFs, images, etc.).
- The server is not limited to sending information from databases, and might alternatively return the result of software tools, or data from communications services.

What can you do on the server-side?

2. **Customized user experience:**

- Servers can store and use information about clients to provide a convenient and tailored user experience. For example, many sites store credit cards so that details don't have to be entered again.
- A deeper analysis of user habits can be used to anticipate their interests and further customize responses and notifications, for example providing a list of previously visited locations you may want to look at on a map.

What can you do on the server-side?

3. **Controlled access to content:**
   - Server-side programming allows sites to restrict access to authorized users and serve only the information that a user is permitted to see.
   - Real-world examples include social-networking sites which allow users to determine who can see the content they post to the site, and whose content appears in their feed.

What can you do on the server-side?

4. **Store session/state information:**
   - ➢ Store session/state information Server-side programming allows developers to make use of sessions — basically, a mechanism that allows a server to store information associated with the current user of a site across multiple requests.
   - ➢ Consider a shopping website where users can add items to their cart and proceed to checkout. The server needs to remember the user's cart items and other details as they navigate through different pages of the website.
   - ➢ sessions are managed using session middleware or libraries provided by the programming language or web framework.
   - ➢ Sessions should be implemented securely by using secure cookies, and setting proper session expiration times.

What can you do on the server-side?

5. **Sending notifications:**
   - Servers can send general or user-specific notifications mainly through the website itself or via email, or SMS.
   - A few examples include:
     - Facebook and Twitter send emails and SMS messages to notify you of new communications.
     - Amazon regularly sends product emails that suggest products similar to those already bought or viewed that you might be interested in.
     - A web server might send warning messages to site administrators alerting them to suspicious user activity.

What can you do on the server-side?

6. **Data analysis:**
   - A website may collect a lot of data about users: what they search for, what they buy, what they recommend, how long they stay on each page. Server-side programming can be used to refine responses based on analysis of this data.
   - For example, Amazon and Google both advertise products based on previous searches and purchases.

Web servers and HTTP (a primer)

- Now that you know the purpose and potential benefits of server-side programming, we're going to examine in detail what happens when a server receives a "dynamic request" from a browser. As most website server-side code handles requests and responses in similar ways, this will help you understand what you need to do when writing most of your own code.
- Web browsers communicate with web servers using the HyperText Transfer Protocol (HTTP). When you click a link on a web page, submit a form, or run a search, the browser sends an HTTP Request to the server.

Web servers and HTTP (a primer)

- HTTP Request includes:

1. A URL identifying the target server and resource (e.g. an HTML file, a particular data point on the server, or a tool to run).
2. A method that defines the required action (for example, to get a file or to save or update some data).
3. Additional information can be encoded with the request.

Web servers and HTTP (a primer)

- The different methods/verbs and their associated actions are:

1. **GET**: Get a specific resource (e.g. an HTML file containing information about a product, or a list of products).
2. **POST**: Create a new resource (e.g. add a new article to a wiki, add a new contact to a database).
3. **HEAD**: Get the metadata information about a specific resource without getting the body like GET would. You might for example use a HEAD request to find out the last time a resource was updated, and then only use the (more "expensive") GET request to download the resource if it has changed.

Web servers and HTTP (a primer)

- The different methods/verbs and their associated actions are:

4. **PUT**: Update an existing resource (or create a new one if it doesn't exist).
5. **DELETE**: Delete the specified resource.
6. TRACE, OPTIONS, CONNECT, PATCH: These verbs are for less common/advanced tasks, so we won't cover them here.

Web servers and HTTP (a primer)

- Additional information can be encoded as:

1. **URL parameters**: GET requests encode data in the URL sent to the server by adding name/value pairs onto the end of it — for example http://example.com?name=Fred&age=11. You always have a question mark (?) separating the rest of the URL from the URL parameters, an equals sign (=) separating each name from its associated value, and an ampersand (&) separating each pair. URL parameters are "insecure" as they can be changed by users and then resubmitted. As a result URL parameters/GET requests are not used for requests that update data on the server.

Web servers and HTTP (a primer)

- Additional information can be encoded as:

2. **POST data**: POST requests add new resources, the data for which is encoded within the request body.
3. **Client-side cookies**: Cookies contain session data about the client, including keys that the server can use to determine their login status and permissions/accesses to resources.

Ch1 Introduction (Part 3)

Server Side Fundamentals

Doing other work, Returning something other than HTML

- A Web Application's job is to receive HTTP requests and return HTTP responses. While interacting with a database to get or update information are very common tasks, the code may do other things at the same time, or not interact with a database at all.
- A good example of an additional task that a Web Application might perform would be sending an email to users to confirm their registration with the site. The site might also perform logging or other operations.
- server-side website code has the flexibility to generate and return various types of content beyond HTML, including text files, PDFs, CSVs, structured data formats like JSON or XML, and custom file formats. This flexibility allows web applications to serve a wide range of content tailored to the needs of the application and its users.

Doing other work, Returning something other than HTML

- This is especially relevant for websites that work by fetching content from the server using JavaScript and updating the page dynamically, rather than always loading a new page when new content is to be shown.
- Imagine you're building a social media platform where users can post updates, like posts, and comment on them.
  - **Initial Page Load:**
    - When the user first visits the platform, the server-side code generates the HTML for the initial page, including the layout, navigation, and perhaps the user's profile information.
    - Additionally, the server-side code may also provide some initial post data (e.g., the latest posts from people the user follows) embedded within the HTML or as JSON data.

Doing other work, Returning something other than HTML

- **Dynamic Updates with JavaScript:**
  - After the initial page load, JavaScript takes over to fetch additional post data from the server as needed. For example, when the user scrolls down the page, JavaScript detects this action and makes a request to the server to fetch more posts.
  - The server-side code responds to these requests dynamically, generating and returning JSON data containing the requested posts.
- **Rendering Dynamic Content:**
  - Once the JSON data is received by the client-side JavaScript, it dynamically updates the page to display the newly fetched posts.
  - This update might involve rendering HTML elements for each post, including the post text, author information, comments, likes, etc.

Web frameworks simplify server-side web programming

- Server-side web frameworks (a.k.a. "web application frameworks") are software frameworks that make it easier to write, maintain and scale web applications. They provide tools and libraries that simplify common web development tasks, which will be listed in next slides.

❖ **A few good web frameworks:**

- The server-side frameworks below represent a few of the most popular available at the time of writing. All of them have everything you need to be productive — they are open source, are under active development, have enthusiastic communities creating documentation and helping users on discussion boards, and are used in large numbers of high-profile websites.

Web frameworks simplify server-side web programming

1. **Django (Python)**

- Django is a high-level Python Web framework that encourages rapid development and clean design. Built by experienced developers, it takes care of much of the difficulty of web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

1. **Flask (Python)**

- While minimalist, Flask can create serious websites out of the box. It contains a development server and debugger, and includes support secure cookies, unit testing, and RESTful request sending. It has good documentation and an active community. Flask has become extremely popular, particularly for developers who need to run a web server on a Drone controllers.

Web frameworks simplify server-side web programming

3. **Express (JavaScript)**

- Express is a fast, flexible and minimalist web framework for Node.js (node is an environment for running JavaScript). It provides a robust set of features for web and mobile applications. Express is extremely popular, partially because it eases the migration of client-side JavaScript web programmers into server-side development.

3. **Ruby on Rails (Ruby)**

- Rails follows a very similar design philosophy to Django. Like Django it provides standard mechanisms for routing URLs, accessing data from a database, generating HTML from templates and formatting data as JSON or XML.

Web frameworks simplify server-side web programming

5. **Laravel (PHP)**

- Laravel is a web application framework with expressive, elegant syntax. Laravel attempts to take the pain out of development by easing common tasks used in the majority of web projects. Laravel is accessible, yet powerful, providing tools needed for large, robust applications.

5. **ASP.NET**

- ASP.NET is an open source web framework developed by Microsoft for building modern web applications and services. With ASP.NET you can quickly create websites based on HTML, CSS, and JavaScript, scale them for use by millions of users and easily add more complex capabilities like Web APIs, forms over data, or real time communications.

Web frameworks simplify server-side web programming

- ASP.NET is an open source web framework developed by Microsoft for building modern web applications and services. With ASP.NET you can quickly create websites based on HTML, CSS, and JavaScript, scale them for use by millions of users and easily add more complex capabilities like Web APIs, forms over data, or real time communications.
- One of the differentiators for ASP.NET is that it is built on the Common Language Runtime (CLR), allowing programmers to write ASP.NET code using any supported .NET language (C#, Visual Basic, etc.). Like many Microsoft products it benefits from excellent tools (often free), an active developer community, and well-written documentation.
- ASP.NET is used by Microsoft, Xbox.com, Stack Overflow, and many others.

Web frameworks simplify server-side web programming

7. **Spring Boot (Java)**

- Spring Boot is one of a number of projects provided by Spring. It is a good starting point for doing server-side web development using Java. It can be used for small problems but its strength is building larger scale applications that use a cloud approach.

❖ **How to select a web framework**

- Numerous web frameworks exist for almost every programming language you might want to use (we list a few of the more popular frameworks in the following section). With so many choices, it can become difficult to work out what framework provides the best starting point for your new web application.
- Some of **the factors that may affect your decision** are:

Web frameworks simplify server-side web programming

1. **Effort to learn**: The effort to learn a web framework depends on how familiar you are with the underlying programming language, the consistency of its API, the quality of its documentation, and the size and activity of its community. If you're starting from absolutely no programming experience then consider Django (it is one of the easiest to learn based on the above criteria). If you are part of a development team that already has significant experience with a particular web framework or programming language, then it makes sense to stick with that.
2. **Productivity**: Productivity is a measure of how quickly you can create new features once you are familiar with the framework, and includes both the effort to write and maintain code (since you can't write new features while old ones are broken).

Web frameworks simplify server-side web programming

Many of the factors affecting productivity are similar to those for "Effort to learn" — e.g. documentation, community, programming experience, etc. — other factors include:

2.1 Framework purpose/origin: Some web frameworks were initially created to solve certain types of problems, and remain better at creating web apps with similar constraints. For example, Django was created to support development of a newspaper website, so it's good for blogs and other sites that involve publishing things. By contrast, Flask is a much lighter-weight framework and is great for creating web apps running on embedded devices.

2.2 Opinionated vs. unopinionated: An opinionated framework is one in which there are recommended "best" ways to solve a particular problem.

Web frameworks simplify server-side web programming

Opinionated frameworks tend to be more productive when you're trying to solve common problems, because they lead you in the right direction, however they are sometimes less flexible.

2.3 Batteries included vs. get it yourself: Some web frameworks include tools/libraries that address every problem their developers can think "by default", while more lightweight frameworks expect web developers to pick and choose solution to problems from separate libraries (Django is an example of the former, while Flask is an example of a very lightweight framework). Frameworks that include everything are often easier to get started with because you already have everything you need, and the chances are that it is well integrated and well documented.

Web frameworks simplify server-side web programming

3. **Performance of the framework/programming language:** Usually "speed" is not the biggest factor in selection because even relatively slow runtimes like Python are more than "good enough" for mid-sized sites. The perceived speed benefits of another language, e.g. C++ or JavaScript, may well be offset by the costs of learning and maintenance.
4. **Caching support:** As your website becomes more successful then you may find that it can no longer manage the number of requests it is receiving as users access it. At this point you may consider adding support for caching. Caching is an optimization where you store all or part of a web response so that it does not have to be recalculated on subsequent requests. Returning a cached response is much faster than calculating one in the first place. Web frameworks will have different levels of support for defining what content can be cached.

Web frameworks simplify server-side web programming

5. **Scalability:** Once your website is fantastically successful you will exhaust the benefits of caching and even reach the limits of vertical scaling (running your web application on more powerful hardware). At this point you may need to scale horizontally (share the load by distributing your site across a number of web servers and databases) or scale "geographically" because some of your customers are based a long way away from your server. The web framework you choose can make a big difference on how easy it is to scale your site.
6. **Web security:** Some web frameworks provide better support for handling common web attacks. Django for example sanitizes all user input from HTML templates so that user-entered JavaScript cannot be run. Other frameworks provide similar protection, but it is not always enabled by default.

Web frameworks simplify server-side web programming

- There are many other possible factors, including licensing, whether or not the framework is under active development, etc.
- If you're an absolute beginner at programming then you'll probably choose your framework based on "ease of learning". In addition to "ease of use" of the language itself, high quality documentation/tutorials and an active community helping new users are your most valuable resources. Django (Python) and Express (Node/JavaScript) are the most chosen for teaching server-side web tutorials mainly because they are easy to learn and have good support.

Web frameworks simplify server-side web programming

❖ **What can a web framework do for you?**

- Web frameworks provide tools and libraries to simplify common web development operations. They will make your life a lot easier.
- One of the most important operations they perform is **providing simple mechanisms to map URLs for different resources/pages to specific handler functions**. This makes it easier to keep the code associated with each type of resource separate. It also has benefits in terms of maintenance, because you can change the URL used to deliver a particular feature in one place, without having to change the handler function.

Web frameworks simplify server-side web programming

Imagine you're building a basic web application for managing tasks. The application has the following functionality:

- Display a list of tasks.
- Add a new task.
- Delete an existing task.

Now, let's assume you're using a web framework that allows you to define routes (URL patterns) and map them to specific handler functions in your code.

In this example, you might have the following URL mappings:

/tasks: Displays the list of tasks.

/tasks/add: Adds a new task.

/tasks/delete: Deletes an existing task.

Web frameworks simplify server-side web programming

And corresponding handler functions in your code:

Handler function for /tasks:

This function retrieves the list of tasks from the database and renders a HTML page to display them.

Handler function for /tasks/add:

This function handles the submission of a form to add a new task. It receives the task details from the form submission, inserts the new task into the database, and redirects the user back to the /tasks page to view the updated list of tasks.

Web frameworks simplify server-side web programming

Handler function for /tasks/delete:

This function handles requests to delete a task. It receives the ID of the task to be deleted, removes it from the database, and redirects the user back to the /tasks page to view the updated list of tasks.

So, when a client (such as a web browser) makes a request to a specific URL:

- If the request is to /tasks, the corresponding handler function retrieves the list of tasks and generates the HTML page to display them.
- If the request is to /tasks/add, the handler function processes the form submission to add a new task.
- If the request is to /tasks/delete, the handler function deletes the specified task from the database.